

Instructions



Do you have problems with practice?

No

not yet

How is it, to be a final year student???

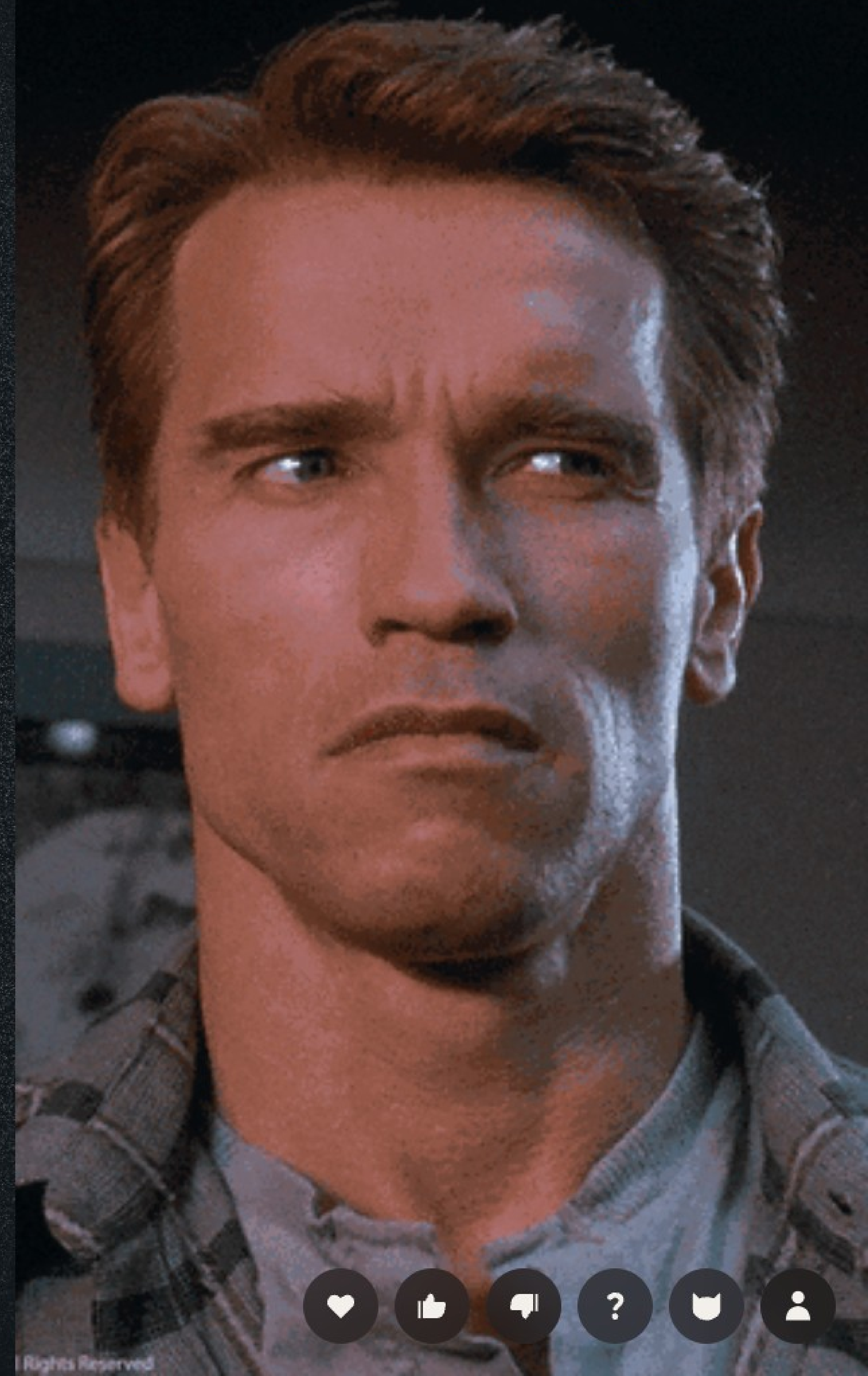


a little hard
tired already

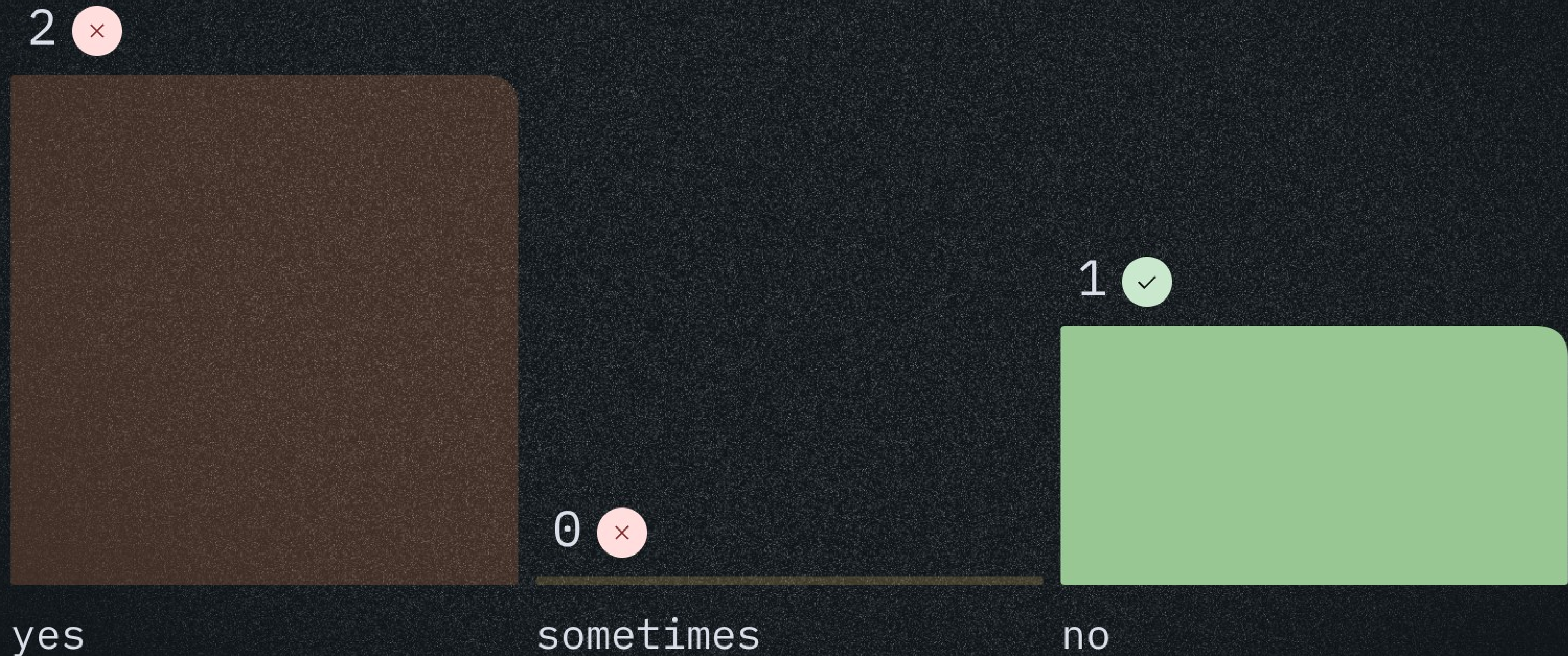
Agenda

- Recall what was learned
- Look at current topic practice
- Discuss theory
- Quiz

Total Recall



Is shallow copy allowed for immutable object?



When do we use singleton pattern?

0 ✗

For every service

3 ✓

When need only one instance
for the object

0 ✗

To reduce boilerplate code

Factory method is a ... pattern.



Which pattern "Allows a client to create families of objects without specifying their concrete classes"?

0 ✕

Prototype

0 ✕

Factory Method

4 ✓

Abstract Factory

Discuss current topic



With Hair



Without Hair

Polar bears have black skin

Have you looked throug the materials?

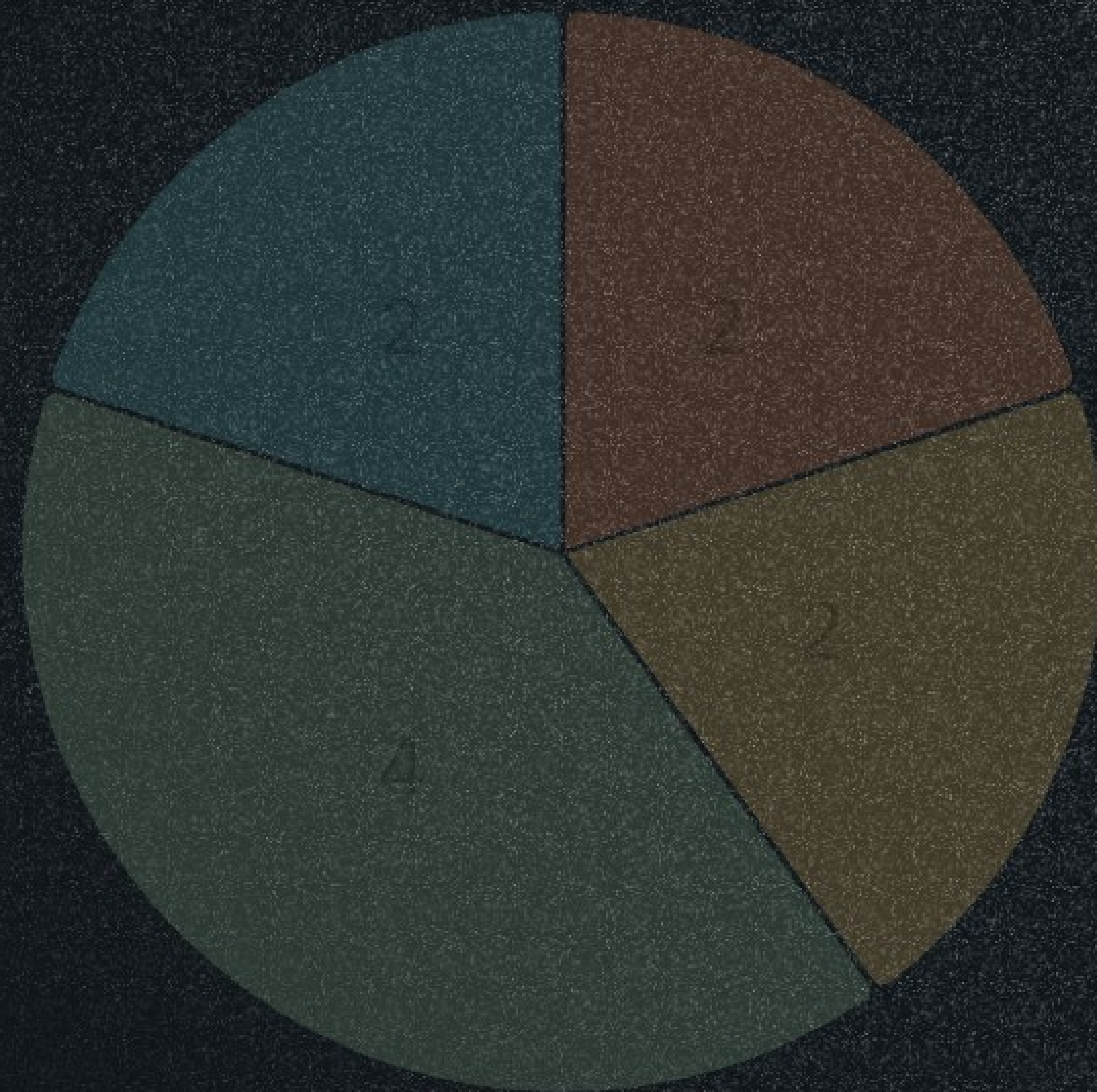


Basic Patterns: Facade

After completing this lesson, you will be able to:

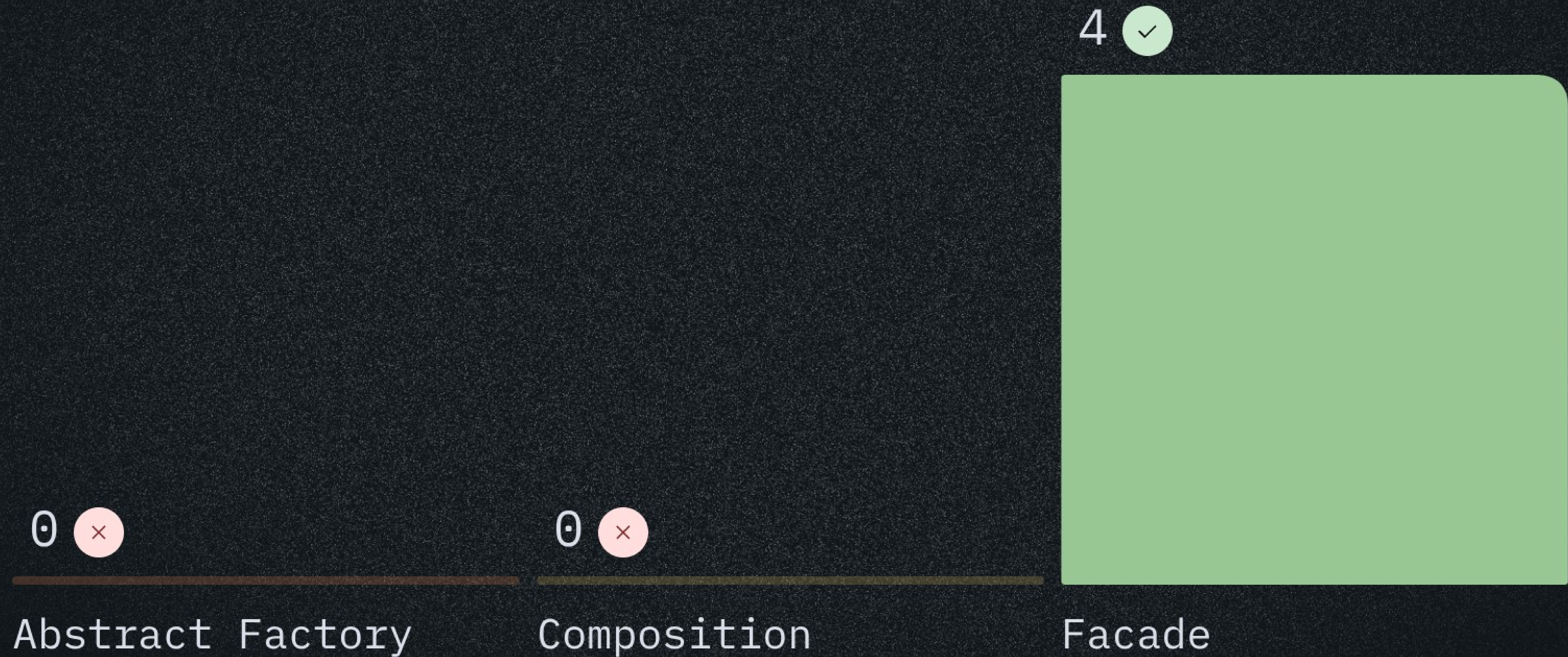
- Explain the feasibility of using the facade pattern to reduce dependencies in a complex class system and make it easier
- Implement the facade pattern to hide system complexity and provide customers with a single point of entry to access its

What is a facade in programming architecture?

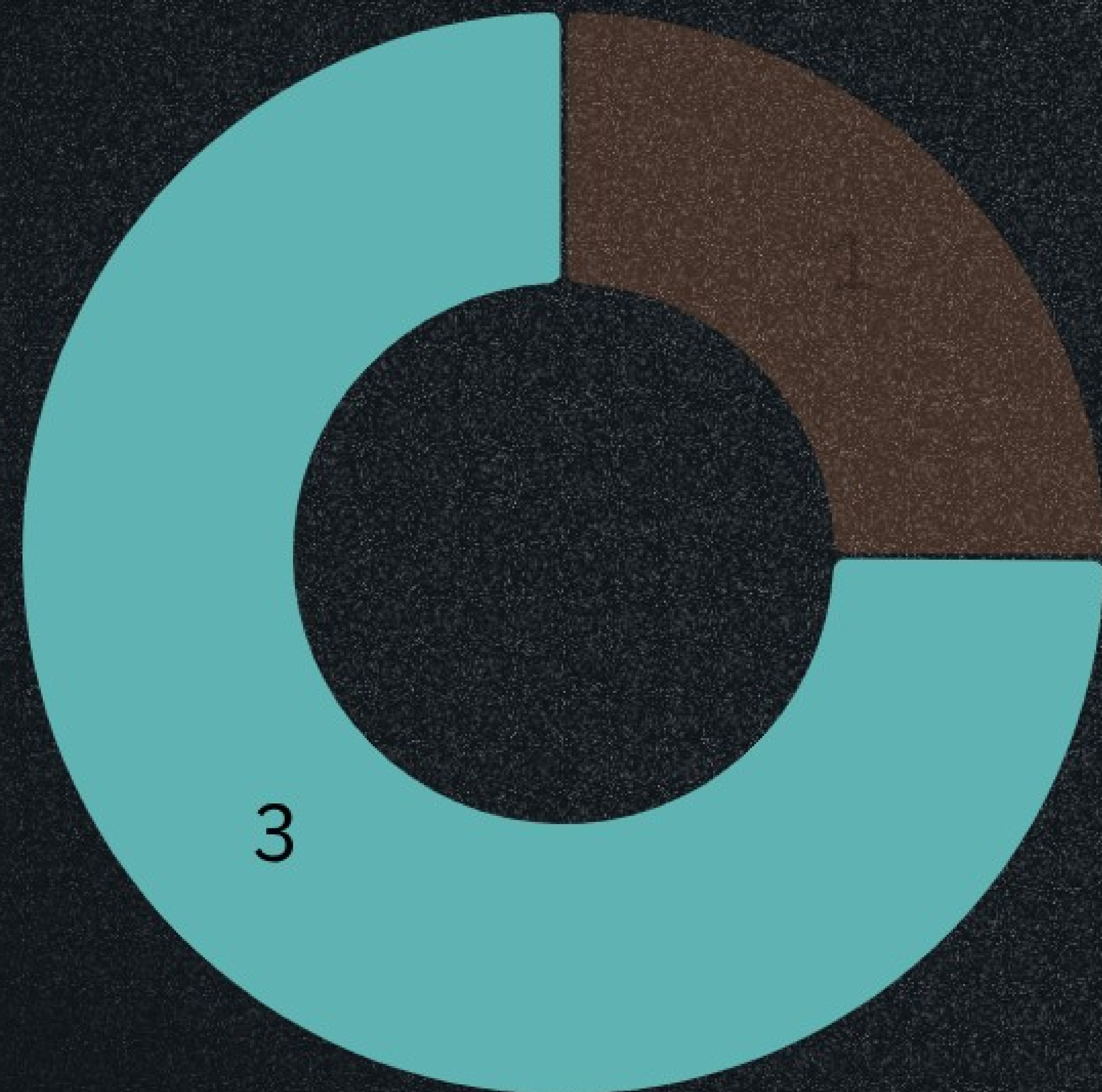


- ☐ 2 A long columned porch that serves as a hallway
- ☐ 2 An open garden surrounded by an enclosed portico
- ☒ 4 A design pattern that provides a simplified interface for a complex subsystem
- ☐ 2 An external wall of a building, often the front wall with the entrance

An object structural pattern that provides a unified interface to a set of interfaces in a subsystem:

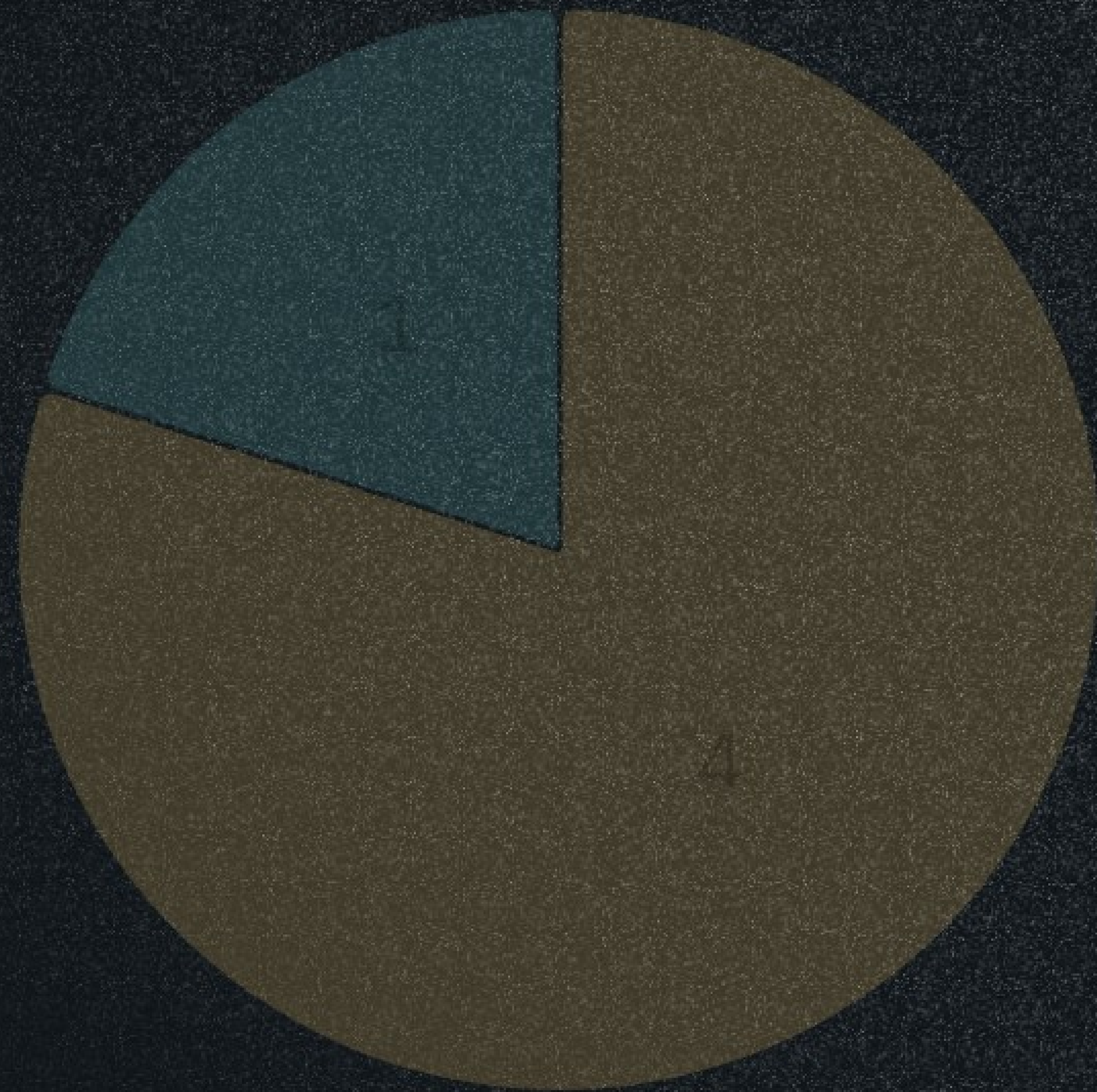


Which of the following is true for facade pattern?



- ☐ 1 a) It is a broker pattern that eases interaction between a client and a sub-system of suppliers by providing a simpler interface to the sub-system ☐
- ☐ 0 b) Systems may have many classes with complex interfaces, but often clients need basic services that can be supplied through simple interfaces ☐
- ☐ 0 c) Broker class, called a façade, provides basic, simplified services to clients by taking upon itself the job of dealing with a complex sub-system ☐
- ☒ 3 d) All of the mentioned ☒

Which of the following is for Facade Pattern?



- 0 a) Interaction is begun by one of the collaborators or a client object outside the collaboration ✗
- 4 b) The object directs the collaboration, with all communication going between it and the colleagues in the interaction ✗
- 0 c) None of the mentioned ✓
- 1 d) All of the mentioned ✗

Pros of Facade pattern

2 ✓



You can isolate your code from the complexity of a subsystem.

1 ✗



A facade can become a god object coupled to all classes of an app.

1 ✗



You can create platform-independent classes and apps.

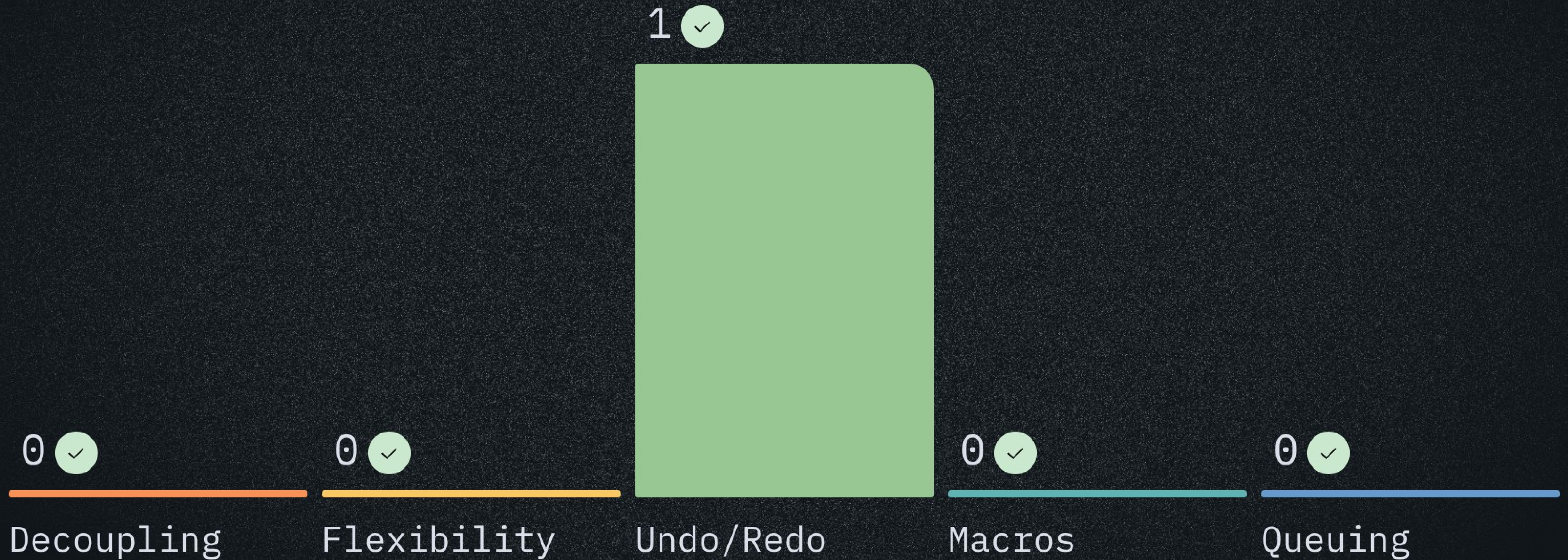
10 mins break

Convenient Patterns: Command

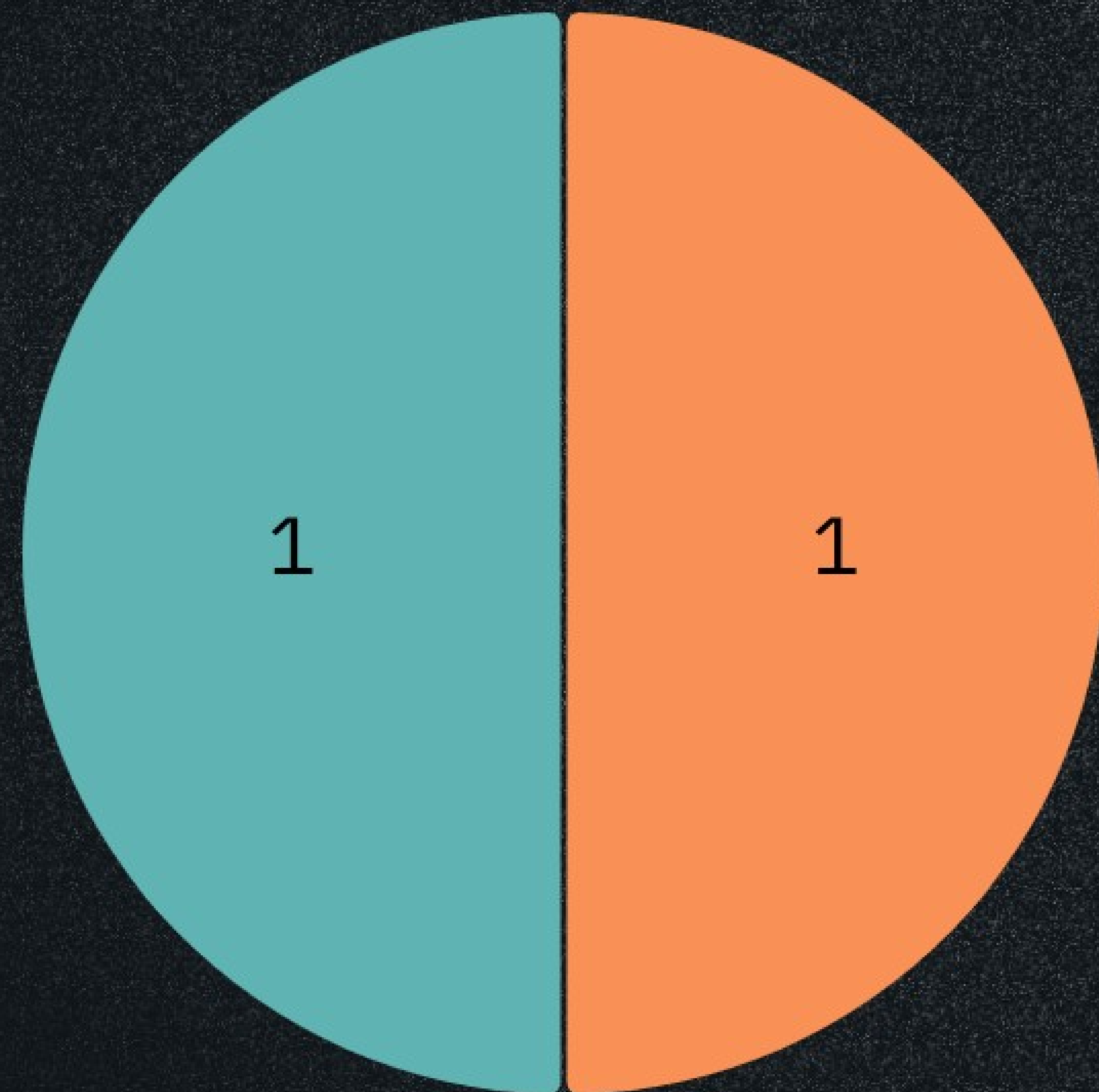
After completing this lesson, you will be able to:

- Explain benefits of using the command pattern to pass request objects as arguments and enqueue, log, and cancel request
- Use the command pattern to encapsulate a request in an object and delegate it to other objects for execution

Benefits of Command pattern



Which of the several aspects of the Command pattern may be varied or elaborated to help achieve particular design goals



- ☒ 1 a) An invoker may accept registration of more than one command ✓
- ☐ 0 b) The invoker may offer operations for un-registering one or more commands ✓
- ☐ 0 c) An invoker may offer several kinds of registrations for different kinds of events ✓
- ☒ 1 d) All of the mentioned ✓

Which of the following are consequences for command pattern?

0 a) Reactor patterns provide event-driven design models



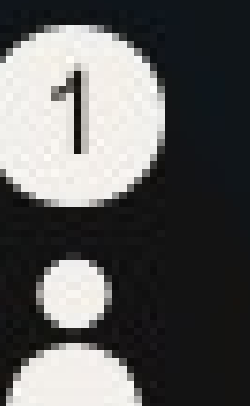
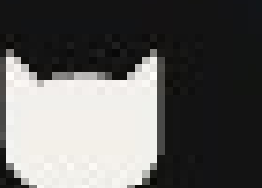
0 b) They decouple clients and targets



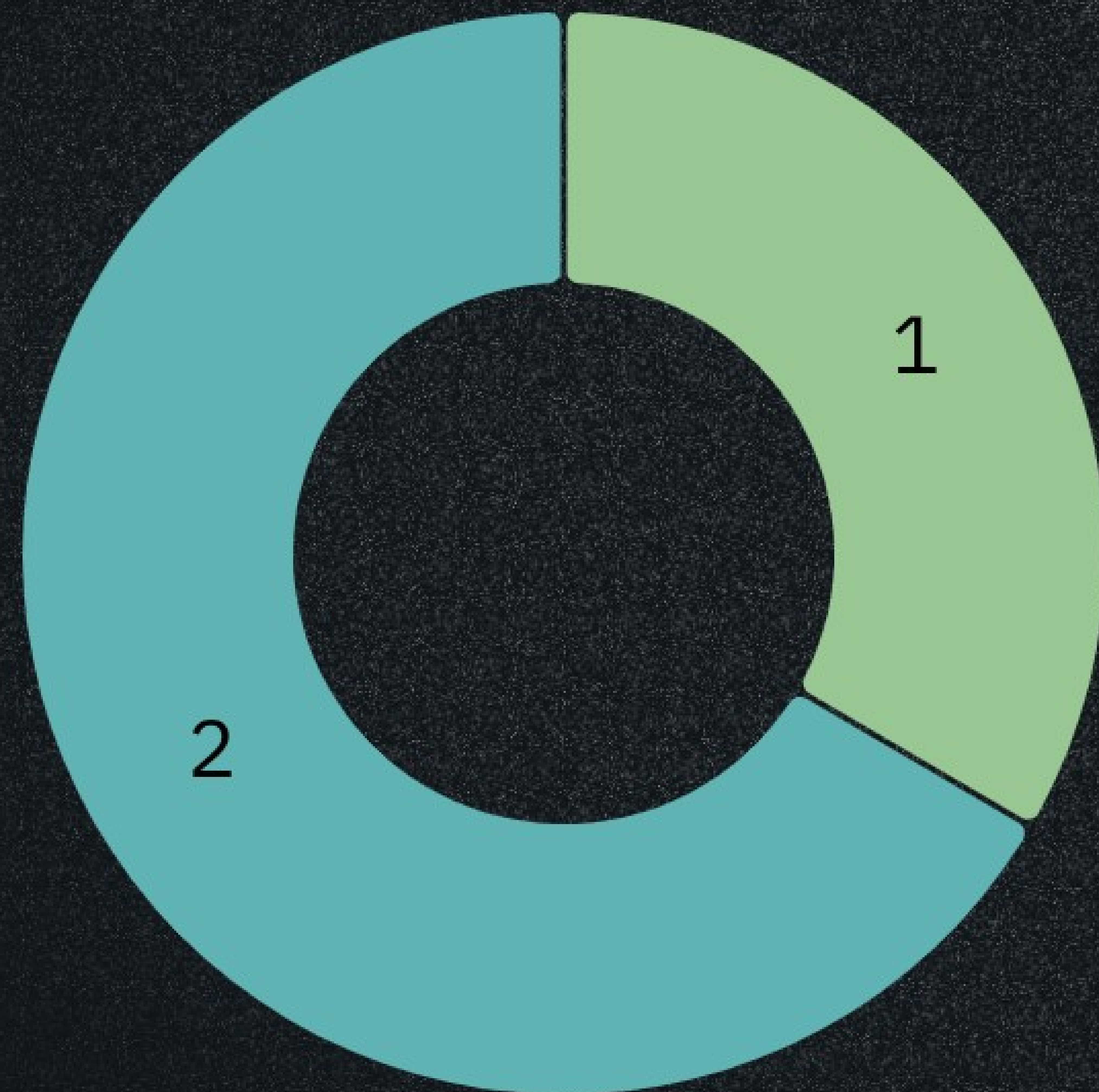
0 c) Encapsulate reactions to event



1 d) None of the mentioned

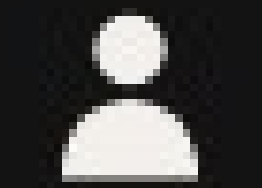
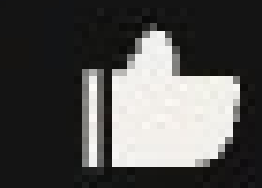


Which of the following are the consequences for Command pattern?



- ☐ 0 a) Event-driven design is facilitated ✓
- ☐ 0 b) Making event-handling code easy to find, change, and reuse ✓
- ☐ 1 c) The invoker is loosely coupled to both the command and client classes, making them easy to change ✓
- ☐ 2 d) All of the mentioned ✓

Quiz



What does the Facade pattern provide?

6 ✓



0 ✗

Direct access to
subsystems

0 ✗

Multiple simultaneous
workflows

0 ✗

Tightly coupled modules

What is the role of a Facade class?

0 ✗

Add complexity to a system

0 ✗

Replace a set of related classes

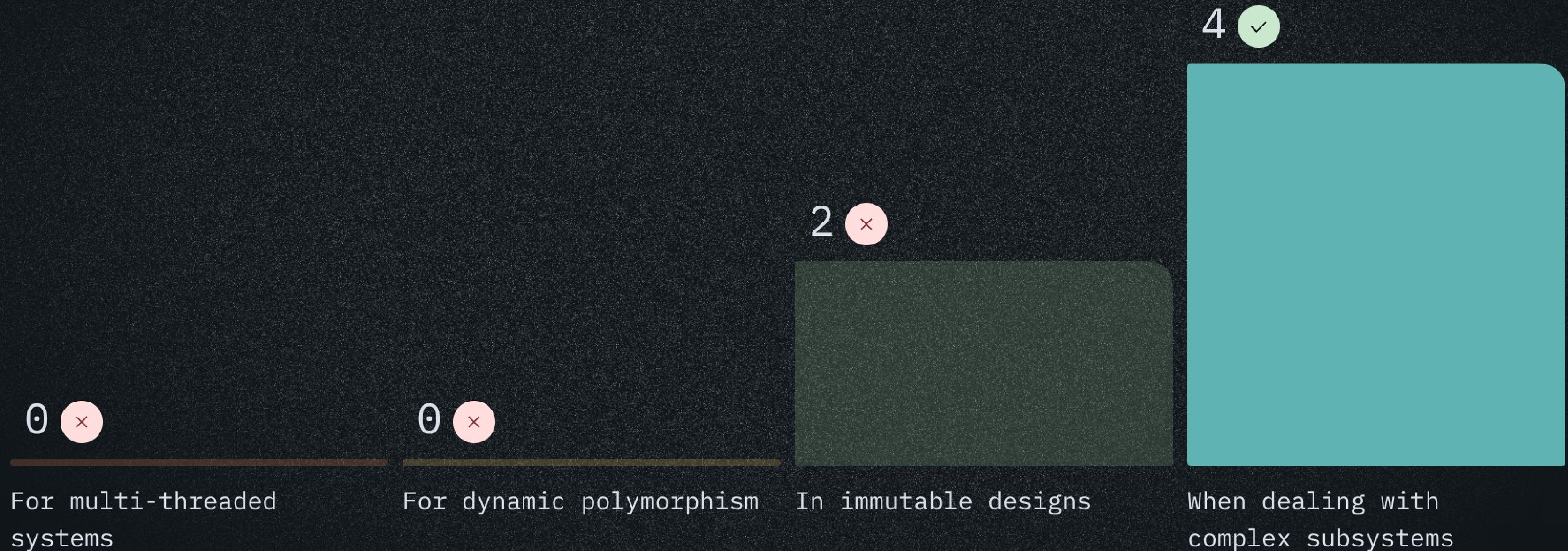
6 ✓

Hide subsystem complexities from the client

0 ✗

Encapsulate object factories

When is the Facade pattern useful?



What principle does Facade comply with?

5 ✓



Encapsulation

1 ✗



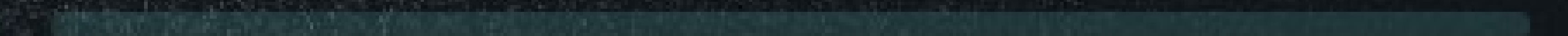
Inheritance

0 ✗

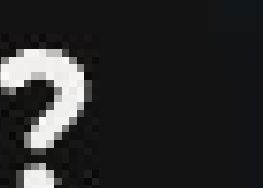
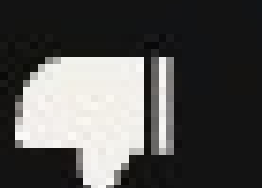


Multi-threading

0 ✗



Generics



Facade promotes which design property?

5 ✓



Loose coupling between
client and subsystems

0 ✗

Multiple inheritance

0 ✗

Object optimization

1 ✗

Redundant classes

A Command object contains?

5 ✓



An action and its receiver

0 ✗

Multiple operations at once

0 ✗

Static method calls

1 ✗

Inherited class definitions

What does the Command pattern achieve?

6 ✓



0 ✗

Simplify user input capture

0 ✗

Optimize runtime performance

0 ✗

Create dependent modules

Command pattern supports which operation?

5 ✓



Undo actions

0 ✗

Static class invocation

0 ✗

Create complex trees

1 ✗

Direct loop termination

What components make up a Command design?

0 ✕

Interface, Inheritance,
Object, Method

0 ✕

Three static class
structures

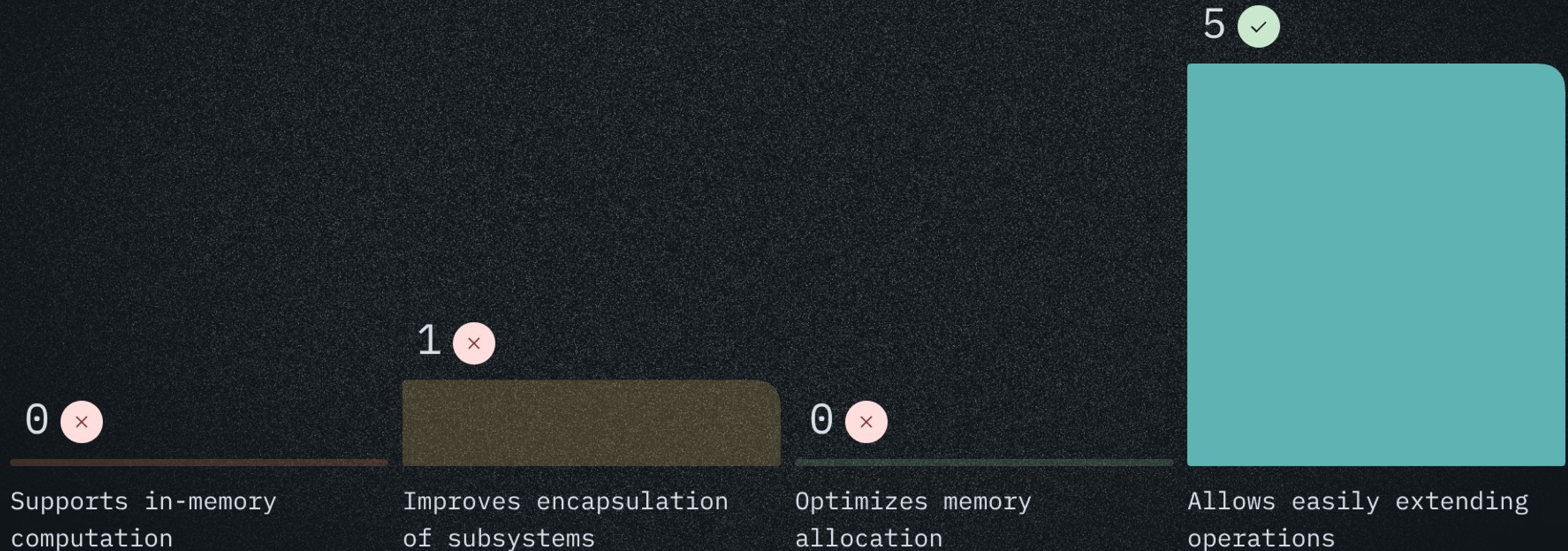
6 ✓

Client, Invoker,
Receiver, Command

0 ✕

Listener, Subject,
Event, Handler

How does Command improve flexibility?



Quiz leaderboard



Q&A part

0 questions

0 upvotes